Mestrado em Engenharia Informática e Multimédia
Visão Artíficial e Realidade Mista
2025/2026

Trabalho Prático 1

Turma: 1N
Trabalho realizado: Miguel Azenha, nº47708
Professor: Pedro Mendes Jorge

12 de Abril de 2026

Conteúdo

1 Introdução **2**

2 Objetivos **2**

3 Enquadramento Teórico **2**

3.1 Visão Computacional 2

3.2 Deteção Facial 3

3.3 Landmarks Faciais 3

3.4 Normalização de Faces 4

3.5 Reconhecimento Facial 4

3.6 Classificação 4

3.7 Realidade Aumentada 5

3.8 Bibliotecas Utilizadas 5

4 Desenvolvimento **5**

4.1 Arquitetura do Sistema 5

4.2 Deteção de Faces 6

4.3 Normalização de Faces 7

4.4 Reconhecimento Facial 9

4.5 Integração de Realidade Aumentada 11

4.6 Execução em Tempo Real 14

5 Trabalho Futuro **15**

6 Conclusão **15**

1 Introdução

No âmbito desta unidade curricular, foi proposto o desenvolvimento de uma aplicação capaz de realizar deteção e reconhecimento facial em tempo real, recorrendo a bibliotecas de visão computacional, como o OpenCV e o MediaPipe. Para além disso, a aplicação deverá integrar técnicas de Realidade Aumentada, permitindo a inserção de objetos virtuais, como óculos, chapéus ou outros elementos, ajustados dinamicamente à posição e orientação das faces detetadas.

O presente trabalho descreve o desenvolvimento de um sistema completo que integra um pipeline composto por três fases principais: deteção de faces, reconhecimento facial e aplicação de elementos de Realidade Aumentada. Serão abordadas as metodologias utilizadas, os algoritmos implementados e os resultados obtidos, bem como uma análise crítica do desempenho do sistema.

2 Objetivos

De forma a atingir este objetivo geral, foram definidos os seguintes objetivos específicos:

- Desenvolver uma aplicação de visão computacional para deteção e reconhecimento facial que permita a inclusão de elementos virtuais alinhados com objetos reais
- Familiarização com a biblioteca OpenCV (Open-Source Computer Vision), entre outras, para programação de aplicações em tempo real
- A aplicação a desenvolver é de Realidade Aumentada (AR), pelo que deverá funcionar em tempo real, e os objetos virtuais terão de estar alinhados com os rostos.
- Os estudantes deverão implementar o pipeline de deteção facial (a), reconhecimento (b) (extração de características e classificação) e aumento (c), escolhendo um dos métodos apresentados

3 Enquadramento Teórico

3.1 Visão Computacional

A Visão Computacional é uma área da inteligência artificial que tem como objetivo permitir que os computadores adquiram, processem e interpretem informação visual do mundo real. Esta área combina técnicas de processamento de imagem, aprendizagem automática e reconhecimento de padrões para extrair informação relevante a partir de imagens ou sequências de vídeo.

Aplicações comuns incluem deteção de objetos, reconhecimento facial, segmentação de imagens e sistemas de navegação autónoma. No contexto deste trabalho, a Visão Computacional é utilizada para identificar e analisar rostos humanos em imagens captadas em tempo real.

3.2 Deteção Facial

A deteção facial consiste no processo de localizar automaticamente faces humanas numa imagem ou num fluxo de vídeo. Este é normalmente o primeiro passo em sistemas de reconhecimento facial.

Existem diversos métodos para a deteção de faces, entre os quais se destacam:

- **Haar Cascade Classifier**: método clássico baseado em características Haar e classificadores em cascata, proposto por Viola e Jones
- **Redes Neuronais Convolucionais (CNN)**: abordagens modernas que utilizam modelos de deep learning para melhorar a robustez da deteção
- **MediaPipe**: framework que utiliza modelos de aprendizagem automática otimizados para deteção de faces e pontos característicos (landmarks) em tempo real

Neste trabalho, foi utilizada a biblioteca **MediaPipe**, que permite não só detetar faces, mas também identificar pontos chave da face, como olhos, nariz e boca, essenciais para etapas posteriores.

3.3 Landmarks Faciais

Os landmarks faciais correspondem a pontos específicos do rosto humano, como os cantos dos olhos, ponta do nariz ou contorno da face. Estes pontos são fundamentais para descrever a geometria da face e permitem realizar operações como alinhamento, rotação e posicionamento de objetos virtuais.

A utilização de landmarks é essencial para garantir que os elementos de Realidade Aumentada são corretamente posicionados e orientados relativamente à face.

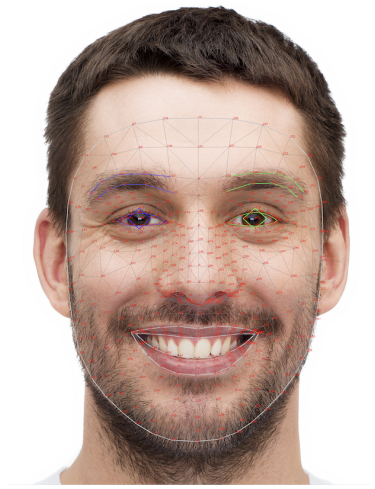

Figura 1: Landmarks faciais

3.4 Normalização de Faces

A normalização de faces é um processo que visa reduzir variações indesejadas nas imagens faciais, tais como rotação, escala ou iluminação. Este passo é fundamental para melhorar o desempenho dos algoritmos de reconhecimento facial.

Tipicamente, a normalização inclui:

- Alinhamento da face com base na posição dos olhos;
- Conversão para escala de cinzentos;
- Redimensionamento da imagem;
- Equalização do histograma.

Estas transformações permitem que diferentes imagens da mesma pessoa apresentem características mais consistentes, facilitando o processo de reconhecimento.

3.5 Reconhecimento Facial

O reconhecimento facial consiste na identificação ou verificação de uma pessoa com base nas suas características faciais. Este processo envolve duas etapas principais: extração de características e classificação.

Entre os métodos mais utilizados destacam-se:

- **Eigenfaces**: baseado na Análise de Componentes Principais (PCA), reduz a dimensionalidade dos dados.
- **Fisherfaces**: utiliza Análise Discriminante Linear (LDA) para melhorar a separação entre classes.
- **LBPH (Local Binary Patterns Histograms)**: método baseado em textura, robusto a variações de iluminação.

Neste trabalho foi utilizado o algoritmo **LBPH**, disponível na biblioteca OpenCV, devido à sua simplicidade e bom desempenho em aplicações em tempo real.

3.6 Classificação

Após a extração de características, é necessário classificar a face como pertencente a um determinado indivíduo. Este processo pode ser realizado através de diferentes técnicas de aprendizagem supervisionada, como **K-Nearest Neighbors (KNN)** ou **Support Vector Machines (SVM)**.

No caso do algoritmo LBPH, a classificação é integrada no próprio modelo, que compara a imagem de entrada com as imagens previamente utilizadas no treino, devolvendo a classe mais provável e um valor de confiança.

3.7 Realidade Aumentada

A Realidade Aumentada (AR) consiste na sobreposição de elementos virtuais sobre o mundo real, em tempo real. Esta tecnologia permite enriquecer a perceção do utilizador através da integração de informação digital no ambiente físico.

No contexto deste trabalho, a AR é utilizada para adicionar objetos virtuais às faces detetadas, como chapéus, óculos ou bigodes. Para tal, é necessário:

- Determinar a posição da face;
- Ajustar a escala do objeto virtual;
- Aplicar rotação de acordo com a orientação da face;
- Sobrepor o objeto utilizando técnicas de blending com máscara (canal alpha).

A correta integração destes elementos depende diretamente da precisão da deteção facial e da qualidade dos landmarks extraídos.

3.8 Bibliotecas Utilizadas

Para a implementação do sistema foram utilizadas as seguintes bibliotecas:

- **OpenCV**: utilizada para processamento de imagem, reconhecimento facial e manipulação de vídeo.
- **MediaPipe**: utilizada para deteção facial e extração de landmarks.
- **NumPy**: utilizada para operações matemáticas e manipulação de arrays.

Estas ferramentas permitem o desenvolvimento eficiente de aplicações de visão computacional com desempenho adequado para execução em tempo real.

4 Desenvolvimento

Neste capítulo é descrito o processo de desenvolvimento da aplicação de deteção e reconhecimento facial com integração de Realidade Aumentada. O sistema foi implementado em Python, recorrendo às já mencionadas.

O pipeline desenvolvido é composto por três fases principais: deteção de faces, reconhecimento facial e aplicação de objetos virtuais.

4.1 Arquitetura do Sistema

O sistema segue uma abordagem sequencial, onde cada frame capturado pela câmara é processado em tempo real. Inicialmente, é realizada a deteção de faces, seguida da

normalização das mesmas. Posteriormente, é efetuado o reconhecimento facial e, por fim, são adicionados objetos virtuais às faces identificadas.

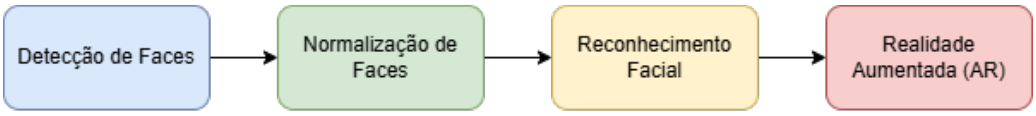

Figura 2: Arquitetura da aplicação

4.2 Deteção de Faces

A deteção de faces foi implementada utilizando a biblioteca MediaPipe, que permite identificar rostos e extrair landmarks faciais com elevada precisão.

Para cada frame de vídeo, a imagem é convertida de BGR para RGB, uma vez que o MediaPipe utiliza este formato. De seguida, é aplicado o modelo de deteção, que devolve os pontos característicos da face.

Listing 1: Funções para extrair landmarks de imagem ou vídeo

```
def get_faces_landmarks_video(frame, timestamp_ms):
    rgb = cv.cvtColor(frame, cv.COLOR_BGR2RGB)
    mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
    faces_landmarks = landmarker_video.detect_for_video(mp_image,
        timestamp_ms).face_landmarks
    return faces_landmarks

def get_faces_landmarks_image(image):
    rgb = cv.cvtColor(image, cv.COLOR_BGR2RGB)
    mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
    faces_landmarks = landmarker_image.detect(mp_image).
        face_landmarks
    return faces_landmarks
```

Com base nos landmarks obtidos, é calculado um retângulo delimitador da face, através da determinação dos valores mínimos e máximos das coordenadas dos pontos. Este retângulo é posteriormente utilizado para desenhar a bounding box e para extrair a região da face.

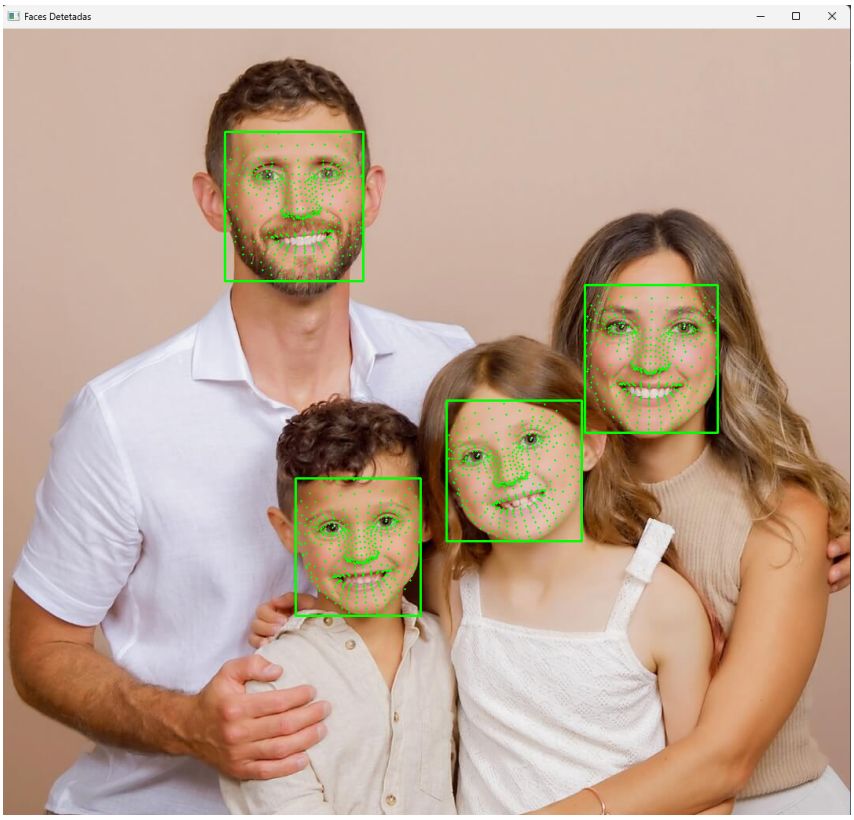

Figura 3: Exemplo de deteção de faces com bounding boxes.

4.3 Normalização de Faces

Após a deteção, as faces são normalizadas de forma a reduzir variações que possam afetar o reconhecimento.

O processo de normalização inclui:

- Alinhamento da face com base na posição dos olhos;
- Rotação da imagem para corrigir inclinações;
- Conversão para escala de cinzentos;
- Redimensionamento para uma dimensão fixa;
- Equalização do histograma.

O alinhamento é realizado calculando o ângulo entre os olhos rodar a imagem para que os olhos fiquem horizontais utilizando a função cv.getRotationMatrix2D, de seguida converter a imagem a tons de cinzento, aplicar um resize de 100x100 e por fim equalizar o histograma dos valores da imagem para reduzir o ruído que provém da captura. Esta etapa é fundamental para garantir consistência entre as imagens do dataset.

Listing 2: Função para normalizar uma face

```

def normalize_face(img,face_position, face_landmarks):
```

```
        ...
        # pontos dos olhos
        left_eye_left_corner = face_landmarks[33]
        left_eye_right_corner = face_landmarks[133]
        right_eye_left_corner = face_landmarks[362]
        right_eye_right_corner = face_landmarks[263]

        # calcular centro de cada olho
        left_eye = ...

        right_eye = ...

        # angulo entre os olhos
        dx = right_eye[0] - left_eye[0]
        dy = right_eye[1] - left_eye[1]
        angle = np.degrees(np.arctan2(dy, dx))

        # calcular centro entre os olhos
        eyes_center = ...

        # rotacao
        M = cv.getRotationMatrix2D(eyes_center, angle, 1.0)
        aligned = cv.warpAffine(img, M, (W, H))
        face_aligned = aligned[y:y+h, x:x+w]
        ...

        # normalizacao final
        normalized_face = cv.cvtColor(face_aligned, cv.
           COLOR_BGR2GRAY)
        # Aplicar um resize de 100x100
        normalized_face = cv.resize(normalized_face, (100, 100))
        # Normaliza os valores
        normalized_face = cv.equalizeHist(normalized_face)

        return normalized_face
```

Figura 4: Faces normalizadas

No enunciado foi referido para utilizar a normalização MPEG-7 cujo resize é de 56x46 no entanto a aplicação apresenta melhor desempenho com um resize de 100x100.

4.4 Reconhecimento Facial

O reconhecimento facial foi implementado utilizando o algoritmo Local Binary Patterns Histograms (LBPH), disponível na biblioteca OpenCV.

Inicialmente, foi criado um dataset de treino contendo várias imagens por indivíduo. Para cada imagem, foi realizada a deteção e normalização da face, sendo posteriormente associada a uma etiqueta correspondente à identidade da pessoa.

Figura 5: Organização do dataset

O modelo foi então treinado utilizando as imagens de treino do dataset, cujas faces foram extraidas e normalizadas e associadas aos respetivos rótulos. O modelo foi treinado usando parâmetros específicos que foram adquiridos através de tentativa e erro para otimizar um melhor valor de confiança que por sua vez melhora o desempenho do sistema.

O algoritmo LBPH foi configurado com os seguintes parâmetros:

- **radius = 1**: Define o raio da vizinhança em torno de cada pixel. Um valor reduzido permite capturar detalhes finos da textura da face, focando-se em padrões locais.
- **neighbors = 8**: Indica o número de pontos considerados à volta do pixel central. Neste caso, são utilizados 8 vizinhos, permitindo uma descrição adequada da textura local com baixo custo computacional.
- **grid_x = 8**: Divide a imagem em 8 regiões na horizontal. Cada região é analisada individualmente, permitindo capturar variações espaciais ao longo da face.
- **grid_y = 8**: Divide a imagem em 8 regiões na vertical. Em conjunto com o parâmetro *grid_x*, resulta numa grelha de 8x8 regiões, totalizando 64 áreas de análise.

Estes parâmetros permitem ao algoritmo extrair características locais de textura e preservar informação espacial da face, contribuindo para um melhor desempenho no reconhecimento facial.

Após o treino,o sistema é capaz de prever a identidade de uma face, devolvendo também um valor de confiança associado à predição.

Listing 3: Treino do modelo

```
dataset_treino_dir = "dataset\\treino"
...

recognizer_parameters = {
    "radius": 1,
    "neighbors": 8,
    "grid_x": 8,
    "grid_y": 8
}
...

print("Treinando o modelo...")
for person_name in os.listdir(dataset_treino_dir):
    print(f"Processando {person_name}...")
    ...

    label_map[current_label] = person_name
    ...

    for img_name in os.listdir(person_path):
        ...

        faces_landmarks = get_faces_landmarks_image(img)
        ...

        for face_landmarks in faces_landmarks:
            face_position = get_face_position(img, face_landmarks)
            normalized_face=normalize_face(img,face_position,
               face_landmarks)

        faces.append(normalized_face)
        labels.append(current_label)
    ...

    current_label += 1

...

recognizer = cv.face.LBPHFaceRecognizer_create(**
   recognizer_parameters)

recognizer.train(faces, labels)

print("Modelo treinado!")
```

Depois de treinar o modelo, o sistema foi testado com imagens de faces conhecidas e desconhecidas do dataset de teste para validar o desempenho do reconhecimento facial.

O modelo retorna, para cada face detetada, a identidade do indivíduo reconhecido e um valor de confiança associado à predição. No caso do algoritmo LBPH, valores de confiança mais baixos indicam uma maior semelhança entre a face analisada e as imagens presentes no conjunto de treino.

De forma a evitar classificações incorretas, foi definido um limiar de confiança. Sempre que o valor de confiança ultrapassa esse limiar, a face é considerada como desconhecida. Esta abordagem permite reduzir erros em situações onde o modelo apresenta incerteza ou quando a face não pertence a nenhuma das classes conhecidas.

No presente trabalho, foi utilizado um limiar de **150**. No entanto, observou-se alguma ambiguidade nos resultados, devido à variabilidade das imagens do dataset. Estas incluem imagens obtidas a partir de diferentes fontes, como fotografias retiradas da internet, capturas realizadas com dispositivos móveis e imagens adquiridas através da câmara do computador, apresentando diferentes condições de iluminação, resolução e qualidade.

Figura 6: Reconhecimento facial com identificação e valor de confiança.

4.5 Integração de Realidade Aumentada

A componente de Realidade Aumentada consiste na sobreposição de objetos virtuais sobre as faces detetadas.

Para cada face identificada, é selecionado um objeto específico (por exemplo, óculos, chapéu ou bigode). A posição do objeto é determinada com base nos landmarks faciais, garantindo um posicionamento adequado.

Para melhorar o realismo, são aplicadas as seguintes operações:

- Redimensionamento do objeto de acordo com o tamanho da face;
- Rotação do objeto com base na orientação da face;
- Aplicação de transparência utilizando o canal alpha.

Para a implementação da componente de Realidade Aumentada, foram desenvolvidas várias funções auxiliares responsáveis pela manipulação e posicionamento dos objetos virtuais sobre as faces detetadas.

Funções para escalar e rodar objeto

A função *rotate_object* permite rodar o objeto virtual de acordo com o ângulo da face, garantindo que este acompanha a orientação do utilizador. Esta operação é realizada através de uma transformação afim.

A função *resize_object* ajusta o tamanho do objeto virtual com base nas dimensões da face detetada, permitindo adaptar corretamente o objeto a diferentes escalas.

Listing 4: Treino do modelo

```
def rotate_object(object, angle):
    h, w = object.shape[:2]
    center = (w // 2, h // 2)
    M = cv.getRotationMatrix2D(center, angle, 1.0)
    rotated_object = cv.warpAffine(object, M, (w, h))
    return rotated_object

def resize_object(face_position, object,scale=(1.0,1.0)):
    x,y,w,h = face_position
    w=int(w*scale[0])
    h=int(h*scale[1])
    object=cv.resize(object,(w,h))
    return object
```

Funções adquirir a poisção do objeto

A função *get_eyes_AngleAndCenter* calcula o ângulo de inclinação da face e o ponto central entre os olhos, sendo utilizada para alinhar corretamente os objetos com a orientação facial.

A função *get_hat_pos* determina a posição ideal para a colocação de um chapéu, utilizando um ponto de referência na parte superior da cabeça e aplicando um pequeno offset.

A função *get_mustache_pos* calcula a posição do bigode com base num ponto específico da face, garantindo o seu correto alinhamento com a região do nariz.

Listing 5: Treino do modelo

```
def get_eyes_AngleAndCenter(img, face_landmarks):
    ...
    lx, ly = int(left_eye.x * w), int(left_eye.y * h)
    rx, ry = int(right_eye.x * w), int(right_eye.y * h)
    angle = -np.degrees(np.arctan2(ry - ly, rx - lx))
    center = ((lx + rx) // 2, (ly + ry) // 2)
    return angle, center

def get_hat_pos(img, face_landmarks,offset=0.05):
    ...
    head = face_landmarks[10]
    x, y = int(head.x * w), int((head.y-offset) * h)
```

```
    return (x, y)

def get_mustache_pos(img, face_landmarks):
    position=(int(face_landmarks[2].x*img.shape[1]),int(
        face_landmarks[2].y*img.shape[0]))
    return position
```

Função para adicionar objeto à imagem original

A função *add_ object* é responsável pela sobreposição do objeto virtual na imagem original. Esta função calcula a região de interesse (ROI), recorta a área relevante do objeto e aplica uma operação de blending utilizando o canal alpha, permitindo integrar o objeto de forma realista na imagem.

Listing 6: Treino do modelo

```

def add_object(frame, obj, position):
    result = frame.copy()
    h, w = obj.shape[:2]
    H, W = result.shape[:2]

    x = position[0] - w // 2
    y = position[1] - h // 2

    x1, y1 = max(0, x), max(0, y)
    x2, y2 = min(W, x + w), min(H, y + h)
    ...

    ox1, oy1 = x1 - x, y1 - y
    ox2, oy2 = ox1 + (x2 - x1), oy1 + (y2 - y1)

    overlay_crop = obj[oy1:oy2, ox1:ox2]
    ...

    roi = result[y1:y2, x1:x2]
    ...

    bgr = overlay_crop[:, :, :3]
    alpha = overlay_crop[:, :, 3:4] / 255.0
    ...

    roi[:] = (alpha * bgr + (1 - alpha) * roi).astype(np.uint8)
    return result
```

Figura 7: Aplicação de objetos virtuais sobre a face.

4.6 Execução em Tempo Real

O sistema foi desenvolvido para funcionar em tempo real, utilizando a câmara do dispositivo. Para cada frame capturado, são executadas todas as etapas do pipeline, garantindo uma resposta rápida e interativa.

A utilização de bibliotecas otimizadas como o OpenCV e o MediaPipe permitiu alcançar um desempenho adequado para aplicações interativas.

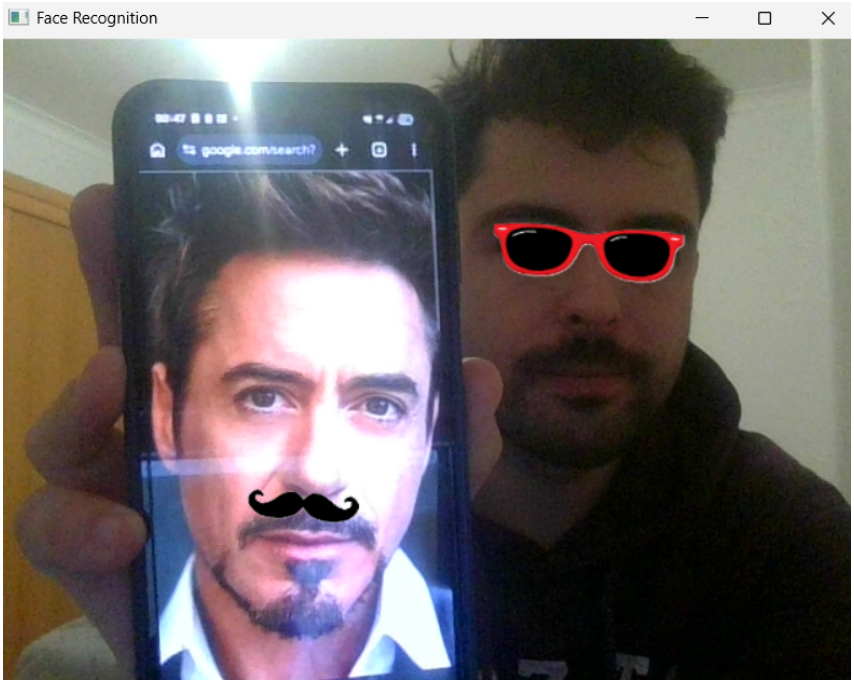

Figura 8: Aplicação de objetos virtuais em tempo real.

5 Trabalho Futuro

Apesar dos resultados obtidos serem satisfatórios, existem várias melhorias que poderão ser exploradas em trabalhos futuros, com o objetivo de aumentar a robustez, precisão e realismo da aplicação desenvolvida.

Uma das principais limitações identificadas está relacionada com a variabilidade do dataset utilizado. A inclusão de um conjunto de dados mais diversificado e equilibrado, com imagens captadas em diferentes condições de iluminação, ângulos e resoluções, poderá contribuir significativamente para melhorar o desempenho do sistema de reconhecimento facial.

Outra possível melhoria consiste na substituição ou complementação do algoritmo LBPH por abordagens baseadas em *Deep Learning*, como modelos de embeddings faciais (por exemplo, redes neurais convolucionais pré-treinadas). Estas técnicas apresentam, geralmente, maior robustez face a variações de iluminação, expressão facial e oclusões.

Adicionalmente, poderá ser otimizado o processo de classificação através da utilização de classificadores externos, como Support Vector Machines (SVM) ou K-Nearest Neighbors (KNN), permitindo uma melhor separação entre classes e uma maior flexibilidade na definição de limiares de decisão.

No que diz respeito à componente de Realidade Aumentada, seria interessante melhorar o realismo dos objetos virtuais, incorporando técnicas mais avançadas de renderização, como ajuste dinâmico de iluminação, sombras e oclusão parcial da face. A utilização de modelos tridimensionais (3D) em vez de imagens bidimensionais também poderia aumentar significativamente a qualidade visual da aplicação.

6 Conclusão

No âmbito deste trabalho, foi desenvolvido um sistema completo de visão computacional capaz de realizar deteção e reconhecimento facial em tempo real, integrando simultaneamente técnicas de Realidade Aumentada. A aplicação implementada demonstrou a viabilidade de combinar diferentes componentes, nomeadamente a deteção de faces, a extração e normalização de características e a classificação, culminando na sobreposição de objetos virtuais de forma coerente com o ambiente real.

A utilização da biblioteca MediaPipe permitiu obter resultados eficazes na deteção de faces e extração de landmarks, essenciais para o correto alinhamento e posicionamento dos objetos virtuais. Por sua vez, o algoritmo LBPH revelou-se uma solução adequada para o reconhecimento facial em tempo real, apresentando um bom equilíbrio entre desempenho e custo computacional.

Os resultados obtidos evidenciam que o sistema é capaz de identificar corretamente diferentes indivíduos e adaptar os elementos de Realidade Aumentada à orientação e dimensão da face. No entanto, verificaram-se algumas limitações, sobretudo relacionadas com a variabilidade das imagens utilizadas no dataset, bem como com condições adversas de iluminação e qualidade de captura, que podem influenciar a precisão do reconhecimento.

De uma forma geral, os objetivos propostos foram atingidos, tendo sido implementado um pipeline funcional e eficiente, capaz de operar em tempo real. Este trabalho permi-

tiu consolidar conhecimentos na área da Visão Computacional e explorar a integração com Realidade Aumentada, evidenciando o potencial destas tecnologias em aplicações interativas e inovadoras.